

Universidad Autónoma de Chihuahua

Facultad de Ingeniería

2K2 / Ingeniería en Computación

Clúster distribuido de MongoDB con Docker Compose

Bases de Datos Avanzadas

Docente: JOSE SAUL DELIRA MIRAMONTES

Carlos González Carrera
Matrícula: 385574
Elio Castillo Reyes
Matrícula: 385770
Jose Fernando Martinez Pacheco
Matrícula: 385795

28 de octubre de 2026

Índice

1. Introducción **1**

2. Objetivo **1**

3. Herramientas y arquitectura **1**

4. Configuración del clúster **2**
4.1. Creación y arranque de los contenedores 2
4.2. Inicialización de los replica sets 2
4.3. Inicio y configuración del router 3

5. Base de datos y clave de particionamiento **4**

6. Operaciones CRUD **5**
6.1. CREATE: insertar un documento 5
6.2. READ: consultar un documento 6
6.3. UPDATE: modificar un documento 6
6.4. DELETE: eliminar un documento 7

7. Comprobación de la distribución **7**
7.1. Verificación de los shards 7
7.2. Distribución de documentos 8

8. Resultados y observaciones **10**

9. Conclusión **10**

1 Introducción

En esta práctica se configuró una base de datos distribuida utilizando MongoDB y Docker Compose. En lugar de guardar toda la información en un solo servidor, se preparó un clúster con tres *shards* que almacenan diferentes partes de la colección. Un router `mongos` recibe las consultas y un servidor de configuración conserva los metadatos de la distribución.

Se trabajó con la base de datos `practica_distribuida` y la colección `usuarios`. Primero se levantaron los contenedores y se configuraron sus *replica sets*; después se realizaron operaciones CRUD y se revisó el reparto de los documentos entre los tres shards. Las capturas incluidas proceden del documento de evidencias elaborado durante la práctica.

2 Objetivo

Implementar y comprobar un clúster de MongoDB sobre Docker con un servidor `configsvr`, tres shards y un router `mongos`; definir la clave de particionamiento de una colección y realizar operaciones de creación, consulta, actualización y eliminación sobre la base distribuida.

3 Herramientas y arquitectura

Se utilizaron Docker, Docker Compose, la imagen `mongo:8.0`, la consola `mongosh` y scripts de JavaScript para automatizar la configuración. La actividad se ejecutó desde la terminal de Windows, como aparece en las capturas.

Servicio	Replica set	Función
`configsvr`	`configReplSet`	Metadatos del clúster
`shard1`	`shard1rs`	Almacenar documentos
`shard2`	`shard2rs`	Almacenar documentos
`shard3`	`shard3rs`	Almacenar documentos
`mongos`	—	Enrutar solicitudes

Cuadro 1: Servicios definidos para el clúster distribuido.

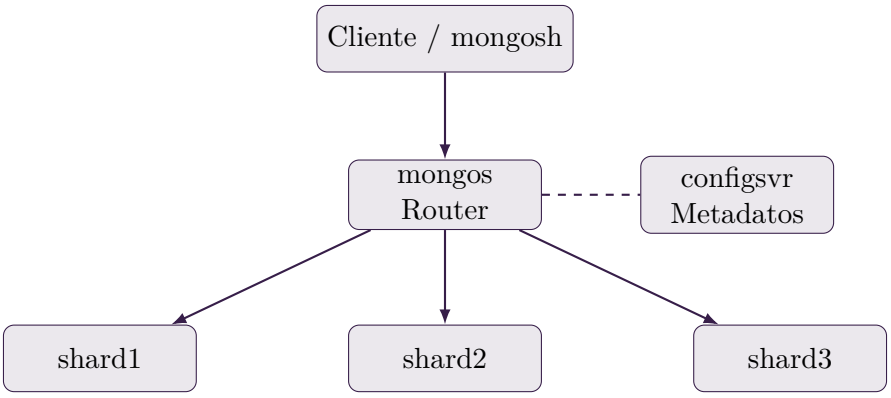


Figura 1: Esquema de los cinco servicios del clúster.

Cada shard se configuró como un replica set de un solo miembro. Esto permite realizar la práctica de particionamiento en una computadora, pero no equivale a un despliegue de alta disponibilidad.

4 Configuración del clúster

4.1 Creación y arranque de los contenedores

En primer lugar se levantaron el servidor de configuración y los tres shards. En la figura siguiente se aprecia que Docker creó los volúmenes y puso en marcha los cuatro contenedores.

```
docker compose up -d configsvr shard1 shard2 shard3
docker ps
```

Listing 1: Inicio de los servidores de MongoDB.

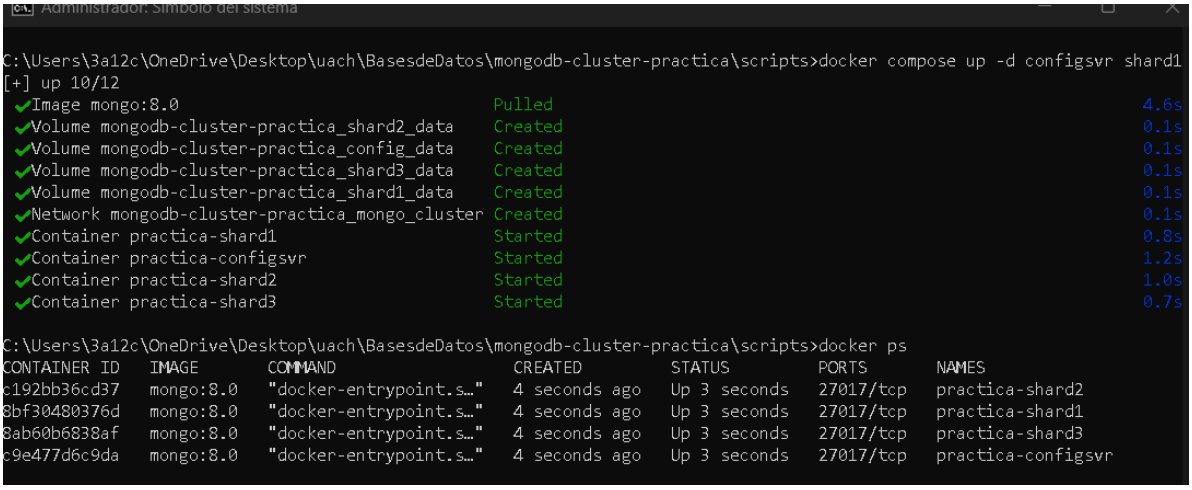


Figura 2: Contenedores iniciales en ejecución y consulta de su estado.

4.2 Inicialización de los replica sets

Con los procesos `mongod` activos se ejecutaron los scripts de inicialización. Cada resultado mostró `ok: 1`, lo que indica que la instrucción fue aceptada.

```
docker compose exec configsvr mongosh --file /scripts/01-configsvr.js
docker compose exec shard1 mongosh --file /scripts/02-shard1.js
docker compose exec shard2 mongosh --file /scripts/02-shard2.js
docker compose exec shard3 mongosh --file /scripts/02-shard3.js
```

Listing 2: Scripts para inicializar los replica sets.

```
C:\Users\3a12c\OneDrive\Desktop\uach\BasesdeDatos\mongodb-cluster-practica\scripts>docker compose exec configsvr mongosh
-file /scripts/01-configsvr.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793244709, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793244709, i: 1 })
}
```

Figura 3: Inicialización del replica set del servidor de configuración.

```
C:\Users\3a12c\OneDrive\Desktop\uach\BasesdeDatos\mongodb-cluster-practica\scripts>docker compose exec shard1 mongosh --
le /scripts/02-shard1.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793244729, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793244729, i: 1 })
}
```

Figura 4: Inicialización del replica set `shard1rs`.

```
C:\Users\3a12c\OneDrive\Desktop\uach\BasesdeDatos\mongodb-cluster-practica\scripts>docker compose exec shard2 mongosh --
le /scripts/02-shard2.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793244767, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793244767, i: 1 })
}
```

Figura 5: Inicialización del replica set `shard2rs`.

```
C:\Users\3a12c\OneDrive\Desktop\uach\BasesdeDatos\mongodb-cluster-practica\scripts>docker compose exec shard3 mongosh --
le /scripts/02-shard3.js
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793244780, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793244780, i: 1 })
}
```

Figura 6: Inicialización del replica set `shard3rs`.

4.3 Inicio y configuración del router

Después se inició `mongos`. En la captura aparecen los cinco contenedores activos. El script de configuración registró los shards en el router y creó la colección particionada.

```
docker compose up -d mongos
docker compose exec mongos mongosh --file /scripts/03-configurar-cluster.js
```

Listing 3: Arranque del router y registro de los shards.

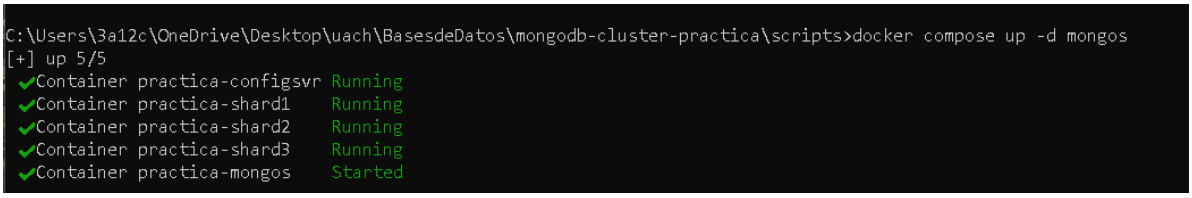


Figura 7: Inicio del router `mongos` junto con los demás servicios.



Figura 8: Ejecución del script de configuración del clúster.

5 Base de datos y clave de particionamiento

Se utilizó la base `practica_distribuida` con la colección `usuarios`. El documento contiene los campos siguientes:

Campo	Tipo previsto	Descripción
`_id`	ObjectId	Identificador interno
`usuarioId`	Número	Identificador del usuario
`nombre`	Cadena	Nombre
`correo`	Cadena	Correo electrónico
`ciudad`	Cadena	Ciudad
`edad`	Número	Edad
`activo`	Booleano	Estado del registro

Cuadro 2: Estructura prevista para la colección `usuarios`.

La clave de particionamiento seleccionada fue `usuarioId` con *hashed sharding*. MongoDB calcula un hash de ese valor y lo usa para asignar los documentos a los rangos que corresponden a cada shard [1]. La configuración está definida en el script `03-configurar-cluster.js`:

```
sh.enableSharding("practica_distribuida");
sh.shardCollection(
  "practica_distribuida.usuarios",
  { usuarioId: "hashed" }
);
```

Listing 4: Definición del shard key.

Se escogió esta clave porque permite repartir identificadores consecutivos sin mantenerlos necesariamente en el mismo shard. Además, las búsquedas por un `usuarioId` específico utilizan la clave de particionamiento.

6 Operaciones CRUD

Las operaciones se realizaron a través del router `mongos`, sobre la colección `usuarios`. Primero se ejecutó el archivo `04-crud.js`, que insertó y consultó usuarios de prueba.

```
docker compose exec mongos mongosh --file /scripts/04-crud.js
```

Listing 5: Ejecución del script CRUD.



Figura 9: Inserción de usuarios y consultas realizadas mediante el script CRUD.

6.1 CREATE: insertar un documento

Se agregó manualmente otro registro con `usuarioId: 200`. La respuesta `acknowledged: true` y el identificador insertado indican que MongoDB aceptó la operación.

```
db.usuarios.insertOne({
  usuarioId: 200,
  nombre: "Carlos",
  correo: "Carlos@Glez",
  ciudad: "juarez",
  edad: "22",
  activo: true
});
```

Listing 6: Inserción individual de un usuario.

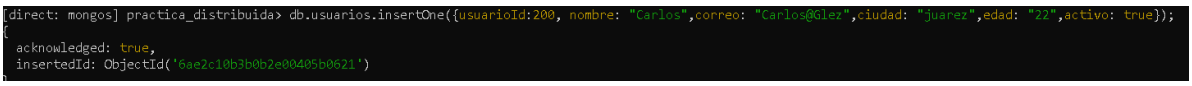


Figura 10: Inserción del usuario con identificador 200.

6.2 READ: consultar un documento

La consulta con `find()` y el campo `usuarioId` recuperó el registro del usuario insertado previamente.

```
db.usuarios.find({ usuarioId: 200 });
```

Listing 7: Consulta por identificador de usuario.

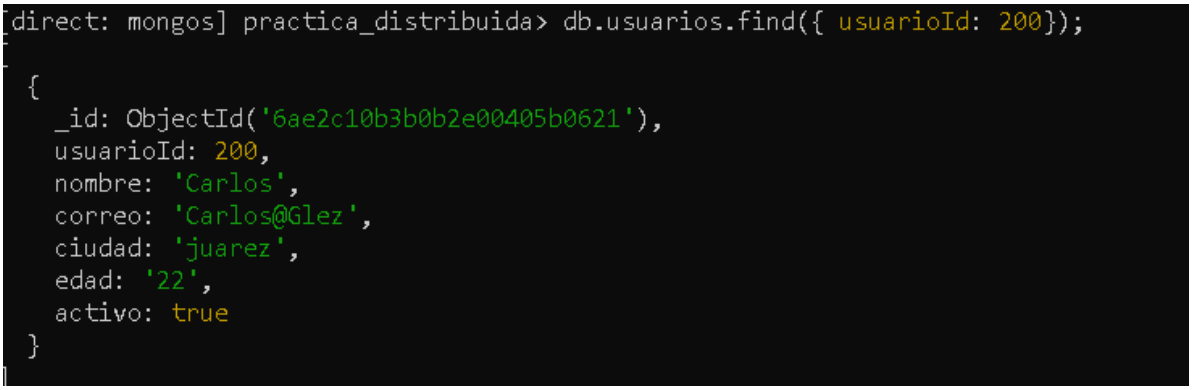


Figura 11: Consulta exitosa del documento con `usuarioId: 200`.

6.3 UPDATE: modificar un documento

Posteriormente se cambió la ciudad del mismo registro a `Chihuahua`. La respuesta mostró `matchedCount: 1` y `modifiedCount: 1`, por lo que se encontró y modificó un documento.

```
db.usuarios.updateOne(
  { usuarioId: 200 },
  { $set: { ciudad: "Chihuahua" } }
);
```

Listing 8: Actualización de la ciudad.

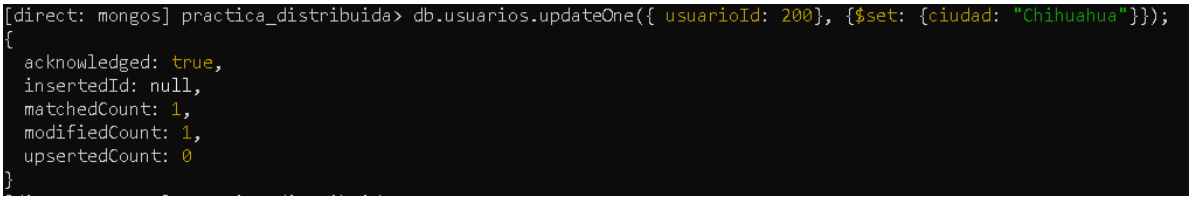


Figura 12: Resultado de la actualización: un documento modificado.

6.4 DELETE: eliminar un documento

Para terminar el CRUD se eliminó el documento `usuarioId: 200`. La respuesta `deletedCount: 1` confirma que se eliminó un registro.

```
db.usuarios.deleteOne({ usuarioId: 200 });
```

Listing 9: Eliminación del usuario.



Figura 13: Resultado de la eliminación de un documento.

Cabe mencionar que la captura de la inserción manual registra `edad` como texto (`"22"`). En el modelo propuesto este campo es numérico; para una aplicación real convendría mantener un tipo de dato consistente.

7 Comprobación de la distribución

7.1 Verificación de los shards

La salida de `sh.status()` muestra los tres replica sets `shard1rs`, `shard2rs` y `shard3rs`. También se observa el router conectado al clúster.

```
sh.status();
```

Listing 10: Consulta del estado del clúster.

```
[direct: mongos] test> sh.status();
shardingVersion
{ _id: 1, clusterId: ObjectId('6ae2be2531bb4c5380a56fe6') }
---
shards
[
  {
    _id: 'shard1rs',
    host: 'shard1rs/shard1:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1793245070, i: 11 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shard2rs',
    host: 'shard2rs/shard2:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1793245070, i: 29 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shard3rs',
    host: 'shard3rs/shard3:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1793245070, i: 52 }),
    replSetConfigVersion: Long('-1')
  }
]
---
active mongoses
[ { '8.0.34': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
```

Figura 14: Estado del clúster: los tres shards registrados en MongoDB.

7.2 Distribución de documentos

Se utilizó una carga adicional de usuarios para comprobar cómo se reparten los documentos. La instrucción `getShardDistribution()` mostró los siguientes valores en la captura final:

```
use practica_distribuida
db.usuarios.getShardDistribution();
```

Listing 11: Consulta de la distribución de usuarios.

Replica set	Documentos	Porcentaje aprox.
shard1rs	106	34.98 %
shard2rs	112	36.96 %
shard3rs	85	28.05 %
Total	**303**	**100 %**

Cuadro 3: Distribución observada de documentos entre los tres shards.

```
[direct: mongos] practica_distribuida> db.usuarios.getShardDistribution();
Shard shard2rs at shard2rs/shard2:27018
{
  data: '15KiB',
  docs: 112,
  chunks: 1,
  'estimated data per chunk': '15KiB',
  'estimated docs per chunk': 112
}
---
Shard shard3rs at shard3rs/shard3:27018
{
  data: '11KiB',
  docs: 85,
  chunks: 1,
  'estimated data per chunk': '11KiB',
  'estimated docs per chunk': 85
}
---
Shard shard1rs at shard1rs/shard1:27018
{
  data: '14KiB',
  docs: 106,
  chunks: 1,
  'estimated data per chunk': '14KiB',
  'estimated docs per chunk': 106
}
---
Totals
{
  data: '40KiB',
  docs: 303,
  chunks: 3,
  'Shard shard2rs': [
    '36.93 % data',
    '36.96 % docs in cluster',
    '137B avg obj size on shard'
  ],
  'Shard shard3rs': [
    '28.07 % data',
    '28.05 % docs in cluster',
    '137B avg obj size on shard'
  ],
  'Shard shard1rs': [
    '34.98 % data',
    '34.98 % docs in cluster',
    '137B avg obj size on shard'
```

Figura 15: Salida de `getShardDistribution()` con 303 documentos en tres shards.

La cantidad de documentos no es exactamente igual entre los tres servidores, pero la salida confirma que existen registros en cada uno de ellos. Por ello, la prueba demuestra que la colección no está almacenada únicamente en un nodo.

8 Resultados y observaciones

Se comprobó que Docker pudo iniciar los cinco servicios y que los scripts de inicialización devolvieron resultados satisfactorios. Las capturas de las operaciones CRUD muestran una inserción reconocida, una consulta con resultados, una modificación y una eliminación efectivas.

Durante el arranque de `mongos` se presentó inicialmente un conflicto con el puerto del equipo anfitrión. Se resolvió ajustando el puerto publicado por Docker Compose, sin cambiar el puerto interno `27017` utilizado por el router. Asimismo, por tratarse de un laboratorio, los replica sets tienen un solo miembro; esta estructura no proporciona tolerancia a fallos si uno de los shards se detiene.

9 Conclusión

En la práctica se configuró un clúster distribuido de MongoDB con Docker Compose y se conectaron el servidor de configuración, los tres shards y el router `mongos`. También se definió una clave de particionamiento `hashed` para la colección `usuarios`.

Las capturas permiten comprobar que las operaciones CRUD funcionaron y que los 303 documentos visibles al final de la prueba quedaron repartidos entre `shard1rs`, `shard2rs` y `shard3rs`. La actividad sirvió para entender la función de cada nodo y cómo MongoDB permite consultar una colección distribuida desde un solo punto de acceso.

Referencias

[1] MongoDB, *Shard Keys*. Documentación oficial. `https://www.mongodb.com/docs/manual/core/sharding-shard-key/`.

[2] MongoDB, *db.collection.getShardDistribution()*. `https://www.mongodb.com/docs/v8.0/reference/method/db.collection.getsharddistribution/`.

[3] Docker, *Docker Compose overview*. `https://docs.docker.com/compose/`.